Project Title
AI Interpreter Workbench: Realtime API vs. Cascade Pipeline

Problem Statement
Boostlingo connects people who need language interpretation with professional human interpreters. AI now offers two distinct architectural patterns for live interpretation, direct voice-to-voice models (OpenAI Realtime API) and composable STT→Translation→TTS cascade pipelines and the trade-offs between them on latency, quality, cost, and operational control are not obvious until both are built. We need an engineer who can build both, instrument them, and form a defensible opinion on when each architecture fits.

Business Context & Impact
Business Context
Live interpretation is Boostlingo's core product. As AI becomes more viable for live interpretation, our architectural choices will determine cost per minute, latency floor, vendor lock-in, and our ability to offer differentiated quality on uncommon language pairs. The industry is split between vertically-integrated voice-to-voice models (less control, lower latency) and cascade pipelines (more control, more moving parts to operate). We need to understand which fits which use case to inform product and platform investment over the next 12–18 months.

Key Impact Metrics
End-to-end latency (ms), cost per minute, interpretation quality (subjective + WER), provider flexibility, time-to-onboard a new language pair

Technical Requirements
Required Programming Languages
Candidate's choice — preferred: .NET / C# (backend), TypeScript (frontend). Python is acceptable. Candidate should explain their choice.

AI/ML Frameworks
OpenAI Realtime API (model: gpt-realtime) required for Realtime mode. Cascade mode providers are candidate's choice e.g., OpenAI / Deepgram / AssemblyAI / Soniox for STT; OpenAI / Anthropic Claude / DeepL for translation; OpenAI TTS / ElevenLabs / Azure Speech / Polly for TTS.

Development Tools

Agentic coding assistant required (Claude Code, Codex, Cursor agent mode, or equivalent, tab-completion only is not sufficient). Git required. Web framework, package manager, and build tooling are candidate's choice.

Cloud Platforms
Optional. Deployment encouraged but not required. Preferred if deploying: AWS. Local-only with clear setup instructions is fine.

Other Specific Requirements
Provider abstractions in cascade mode so STT/translation/TTS providers can be swapped without rewriting the app. Streaming throughout the cascade pipeline (no full-utterance blocking). AGENTS.md or CLAUDE.md documenting how the candidate directed their coding agent. Git history reflecting iterative development. Per-stage latency instrumentation visible in the UI.

Success Criteria
Functional Requirements (Must-Haves)
1. Browser-based SPA with microphone capture and audio playback
2. Realtime mode using OpenAI Realtime API (gpt-realtime) - voice in, voice out
3. Cascade mode using a composed STT → Translation → TTS pipeline with streaming
4. UI toggle to switch between modes mid-session or pre-session
5. Language pair selection (minimum: English ↔ Spanish)
6. Live transcripts showing both source and target text as they're produced
7. Per-stage latency display visible to the user
8. Comparison write-up (1–2 pages) covering latency, quality, cost, controllability, and a recommendation for which mode fits which scenario

Code Quality Expectations
Clean separation between mode-specific transport and mode-agnostic UI. Provider abstractions (interfaces or equivalent) for STT/TTS/translation - swapping a provider should be a contained change. Targeted tests on the cascade pipeline and provider boundaries; full coverage not required, critical paths must be tested. Error handling for provider failures (rate limits, timeouts, empty results, mic permission denied). README with setup, run, and architecture overview. AGENTS.md/CLAUDE.md describing agent usage. Commits scoped to logical units of work with meaningful messages - no single "initial commit" dumps.

Performance Benchmarks
Realtime mode: under 1.5s end-to-end perceived latency (speech end → first audio out).

Cascade mode: under 3s end-to-end, target under 2s with full streaming.
Stability: sustain a 5-minute back-and-forth conversation without disconnection, audio drift, or memory leaks.

Time Constraints
3–4 days for the build (~15–20 hours total effort).